W2 Project · When Local Messages Mislead

Advanced Machine Learning for Artificial Intelligence · **Total: 10 points**

Deadline: use the deadline displayed in the SmartClass assignment entry. Submit through that entry; do not place your submission or identifying information in the public course repository.

Starting point: W2 teaching lab and project starter. CPU execution is sufficient. All data are synthetic and generated by your code.

Research question

When can local inference look numerically stable while answering the wrong probabilistic question, and how can a structural intervention in the computation reveal or correct the error?

You will investigate binary pairwise models with spins $s_i \in \{-1, +1\}$:

$$p(s) = \frac{1}{Z} \exp\left(\sum_i h_i s_i + \sum_{(i,j)\in E} J_{ij} s_i s_j\right).$$

Code states `0` and `1` represent spins `-1` and `+1`, respectively. A positive coupling favors agreement; a negative coupling favors disagreement. Couplings are log-potential parameters, not transition probabilities. Parameters are fixed for inference; you are not asked to fit a model to real data.

The provided module contains a small exact enumeration oracle, tree sum-product and a synchronous loopy-BP baseline. You may use these baselines. Merely wrapping them and plotting their outputs does not complete this project. The new exact cycle method and your proposed extension must be your implementation, supported by your own derivation and tests. Do not use a library's general-purpose exact inference solver for those assessed components.

A. State a claim before searching for a counterexample

Create a short prediction ledger **before** the parameter sweep. Record a mathematical claim about convergence or accuracy, the mechanism you expect, the metric and tolerance that would refute it, and the models you plan to test. Save the ledger with a timestamp or version-control revision. A timestamp supports your research record; it is not proof that an idea was independently conceived.

Design two small model families, with 4–12 nodes so the complete-joint oracle is feasible. Include a tree control, a single-cycle family, and at least one frustrated cycle (an odd number of negative coupling signs around the cycle). Choose and justify the field patterns and coupling ranges. Include at least three field/coupling settings and three seeds where randomness is used. Avoid a study in which every zero-field marginal is exactly one-half by symmetry: that can hide joint errors and misleadingly suggest that inference is easy.

For each model, compare exact one-node marginals with the loopy baseline under old-message weights 0, 0.3 and 0.6, a residual tolerance of 10^{-9} and at most 1,000 iterations. Record convergence status, final residual, maximum one-node total variation error, update count and wall time. Never discard a nonconverged run from the summary. Report at least one individually inspectable failure, including its full graph, fields and couplings. Explain what a failed search would and would not establish; you are not graded for inventing a failure that did not occur.

B. Derive and implement an exact cycle correction

Choose a single-cycle model and one cycle node C. Conditioning on $C = c$ and absorbing its incident factors into its neighbors leaves a chain on the other nodes. Derive a method that uses tree inference on the reduced graph and reconstructs the original one-node marginals.

Your derivation must identify:

- the reduced factors, including which terms become constants;
- what each reduced partition function represents;
- the correct relative weight of each value of C;
- how to recover the marginal of C and of every remaining node;
- the difference between an unnormalized evidence mass and a probability;
- the computational and memory costs in terms of node count and state count.

Implement `cycle_cutset_exact(model, cut_node)` returning at least `marginals` and `log_z`. Do not call enumeration inside it. Validate against enumeration on at least ten small models, including unequal node fields, mixed-sign couplings and a nonsymmetric pairwise table. Agreement in both marginals and log normalizer should be within 10^{-8} in float64. State if an extreme test needs a justified relaxed tolerance.

Also test that multiplying any factor by a positive constant leaves normalized marginals unchanged while changing the log normalizer by the known amount. An algorithm that averages normalized conditional answers with equal weights will often look plausible on symmetric examples; your tests must detect that failure. The provided `condition` method clamps values but deliberately retains graph edges; you must perform the actual reduction, not feed the unchanged cyclic graph into the tree solver.

Demonstrate your method on a 60-node cycle where enumeration is intentionally unavailable. This is a scalability demonstration, not a claim that you have obtained an independent exact oracle at that size. Validate using structural identities, alternative cut-node choices and numerical invariances. All messages and evidence weights must remain numerically stable.

C. Make and test your own extension

Choose a computational question that your cycle correction does not already settle. Propose an extension or adaptive decision rule motivated by a derivation or an explicit model mechanism. Examples of directions include selecting a cutset under a budget, detecting when a residual is an unreliable stopping signal, or deciding where extra exact computation is most valuable in a graph with more than one cycle. These are starting directions, not prescribed solutions.

Define precisely what your method computes, what it assumes and whether its output is exact or approximate. A parameter sweep alone is not an extension. If you choose an adaptive rule, state its inputs, output, compute budget and selection criterion. If you introduce a learned rule, keep training/tuning cases separate from evaluation cases and explain why learning is warranted.

Freeze the method before evaluating on **five additional seeds and at least one graph structure or coupling regime not used to tune it**. Compare against the supplied loopy baseline and an appropriate simpler control at a stated computation budget. Use the small oracle when feasible; report timing distributions rather than one convenient time. Include an ablation that removes the ingredient you believe matters. Your extension need not win everywhere. Credit depends on a testable idea, correct implementation, honest tradeoffs and a defensible explanation of success or failure.

D. Defend the probabilistic reasoning

Include a concise written response to all three questions:

1. Which conditional independence becomes available after conditioning and reducing the cycle, and how does it justify your code's factor manipulations?
2. Why does a small normalized-message residual fail to certify small posterior error? Relate your answer to one actual experiment, including a case where they agree or disagree.
3. What evidence would change your current conclusion about your extension? Identify one limitation of your evaluation and design a new test that addresses it.

Make the reasoning specific to your own model, numerical results and code. You should be able to explain every submitted component and any outside assistance. Generic explanations detached from the submitted experiments do not satisfy this requirement.

Submission contract

Submit one ZIP containing:

- `report.pdf`: a concise research report, approximately 4–6 pages excluding references. Include the prediction ledger, derivation, graph diagrams, main results, ablation, limitations and written defense.
- `project.ipynb`: an executed notebook that runs from a fresh CPU runtime, with visible versions/seeds and no hidden local-file dependencies.
- your reusable implementation files, plus `README.md` with one clear reproduction command or top-to-bottom notebook instructions;
- `results.csv` or `results.json` containing every evaluated configuration, its seed, exact/approximate status, residual, error, iteration/update count and timing;
- tests for correctness, impossible/degenerate cases that you claim to support, and factor-rescaling invariance;
- `assistance.md`: identify collaborators, code sources and any generative-AI assistance, distinguishing what you reused from what you derived, implemented and checked yourself. Cite licensed external code appropriately and follow the course's existing academic-integrity policy.

Do not embed credentials or personal datasets. The starter intentionally leaves assessed methods unfinished; do not submit it unchanged. Preserve the experiment ledger even if your first claim turns out to be false. No additional late-submission rule is introduced here; the existing course policy and SmartClass settings apply.

Rubric · 10 points

Criterion	Points	Evidence for full credit	Partial-credit guidance
Probabilistic derivation and structural reasoning	2	Correct reduced factors, evidence weights, normalizer and reconstruction; conditional independence and complexity justified	1: main idea correct but normalization or scope incomplete; 0: no coherent derivation
Exact correction implementation and independent validation	2	Working cycle method; ten varied oracle checks, nonsymmetric potentials, scale invariance and 60-node validation	1: partially correct implementation or restricted tests; 0: only supplied baseline/oracle calls
Counterexample study and experimental discipline	2	Prior claim, tree control, cycle/frustration study, all outcomes recorded, accuracy separated from convergence	1: informative results with missing controls or selective reporting; 0: unsupported figures/claims
Original extension and held-out evaluation	2	A justified new choice, frozen method, five new seeds plus an unseen regime, fair comparison and ablation	1: meaningful idea with incomplete evaluation; 0: parameter sweep only or no extension
Reproducibility and technical communication	1	Fresh-runtime execution, documented versions/seeds, machine-readable results and assistance/source record	0.5: minor reproducibility gaps; 0: results cannot be reconstructed
Written defense and limitations	1	Three specific answers tied to equations, code and evidence, plus a test that could change the conclusion	0.5: mostly descriptive answers; 0: generic or absent reasoning

The total is always **10**. Within a two-point category, half-point increments may recognize intermediate evidence. A correct, carefully analyzed negative result can receive full credit. Performance improvement without a valid comparison or understanding does not by itself earn full credit.

Reading

Murphy, *Probabilistic Machine Learning: Advanced Topics*, §§4.2.4, 4.3.6, 9.3–9.5; official book resources. The project is an original course assignment built on the W2 lab; no textbook exercise solution is supplied.

Code definitions and further study

API: arguments, results and examples

- PairwiseModel: Finite undirected model represented by log potentials, not fitted parameters.
- enumerate_exact: Enumerate the complete joint as an independent small-model oracle.

- tree_sum_product: Run exact two-pass log-domain sum-product on a connected tree.
- loopy_sum_product: Run synchronous approximate sum-product with probability-space damping.
- max_marginal_tv: Return max_i 0.5*sum_k(abs(p[i,k]-q[i,k])) as a float.